```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from scipy.special import rel_entr

# ==========================================
# 1. DYNAMIC PAYOFF MATRICES (The Weather)
# ==========================================
# Format: (Reward R, Temptation T, Sucker S, Punishment X)
PAYOFFS = {
    'Clear': {'R': 3, 'T': 5, 'S':  0, 'X':   1},  # Safe to speed
    'Rain':  {'R': 3, 'T': 4, 'S': -1, 'X':  -3},  # Slippery, mild penalty
    'Fog':   {'R': 3, 'T': 2, 'S': -2, 'X':  -6},  # Low visibility, severe penalty
    'Snow':  {'R': 3, 'T': 1, 'S': -2, 'X': -10}   # Ice, catastrophic penalty
}

# Strategies: 0=Always Cooperate (AC), 1=Always Defect (AD), 2=Tit-for-Tat (TFT), 3=Random (RAND)
STRATEGIES = ['AC', 'AD', 'TFT', 'RAND']
COLORS = ['blue', 'red', 'green', 'orange']

# Empirical probabilities from Assignment 2 [Cooperate/Comply, Defect/Violate]
EMPIRICAL_DATA = {
    'Clear': np.array([0.582, 0.418]),
    'Rain':  np.array([0.806, 0.194]),
    'Fog':   np.array([0.903, 0.097]),
    'Snow':  np.array([0.951, 0.049])
}

# ==========================================
# 2. EVOLUTIONARY NETWORK SIMULATION
# ==========================================
def run_ipd_simulation(weather, generations=50, N=200):
    # Small-World Network
    G = nx.watts_strogatz_graph(N, k=4, p=0.1)

    # Initialize strategies equally
    strats = np.random.choice([0, 1, 2, 3], size=N, p=[0.25, 0.25, 0.25, 0.25])
    nx.set_node_attributes(G, dict(enumerate(strats)), 'strategy')

    history_freq = {s: [] for s in range(4)}
    payoff = PAYOFFS[weather]

    for gen in range(generations):
        scores = np.zeros(N)

        # Play IPD with neighbors
        for i in G.nodes():
            s_i = G.nodes[i]['strategy']
            for j in G.neighbors(i):
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

```
                    s_j = G.nodes[j]['strategy']

                    # Simplified 1-round interaction
                    move_i = 0 if s_i == 0 else (1 if s_i == 1 else np.random.choice([0, 1]))
                    move_j = 0 if s_j == 0 else (1 if s_j == 1 else np.random.choice([0, 1]))

                    # TFT logic approximation
                    if s_i == 2: move_i = 1 if s_j == 1 else 0
                    if s_j == 2: move_j = 1 if s_i == 1 else 0

                    # Calculate Payoffs
                    if move_i == 0 and move_j == 0: scores[i] += payoff['R']
                    elif move_i == 1 and move_j == 0: scores[i] += payoff['T']
                    elif move_i == 0 and move_j == 1: scores[i] += payoff['S']
                    elif move_i == 1 and move_j == 1: scores[i] += payoff['X']

            # Track Frequencies
            current_strats = list(nx.get_node_attributes(G, 'strategy').values())
            for s in range(4):
                history_freq[s].append(current_strats.count(s) / N)

            # Evolution: Proportional Reproduction
            min_score = np.min(scores)
            if min_score < 0: scores = scores + abs(min_score) + 1

            new_strats = np.zeros(N, dtype=int)
            for i in G.nodes():
                neighbors = list(G.neighbors(i)) + [i]
                neighbor_scores = scores[neighbors]
                if np.sum(neighbor_scores) == 0:
                    probs = np.ones(len(neighbors)) / len(neighbors)
                else:
                    probs = neighbor_scores / np.sum(neighbor_scores)
                chosen_neighbor = np.random.choice(neighbors, p=probs)
                new_strats[i] = G.nodes[chosen_neighbor]['strategy']

            nx.set_node_attributes(G, dict(enumerate(new_strats)), 'strategy')

        return history_freq

    # ==========================================
    # 3. RUN ALL WEATHERS & PLOT (Modified for 4 separate graphs)
    # ==========================================
    print("--- KL DIVERGENCE SCORES ---")
    for weather in ['Clear', 'Rain', 'Fog', 'Snow']:
        freq = run_ipd_simulation(weather)

        # Create a new, standalone figure for each weather condition
        plt.figure(figsize=(8, 5))

        # Plotting
        for s in range(4):
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

```
for s in range(4):
    plt.plot(freq[s], label=STRATEGIES[s], color=COLORS[s], lw=2)

plt.title(f"Strategy Evolution: {weather} Weather")
plt.xlabel("Generation")
plt.ylabel("Fraction $x_k(t)$")
plt.grid(alpha=0.3)
plt.legend(loc='best') # Ensure legend appears on every graph

# KL Divergence Calculation
# Final compliance is sum of AC (0) and TFT (2)
sim_coop = freq[0][-1] + freq[2][-1]
P_sim = np.array([sim_coop, 1 - sim_coop])
P_sim = np.clip(P_sim, 1e-5, 1 - 1e-5) # Avoid div by zero

kl_score = sum(rel_entr(EMPIRICAL_DATA[weather], P_sim))
print(f"{weather}: {kl_score:.4f} (Simulated Compliance: {sim_coop*100:.1f}%)")

# Save and show the individual graph
plt.tight_layout()
plt.savefig(f"evolution_{weather.lower()}.png")
plt.show()
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.